The UNIX-HATERS Wiki

Express/Elan Graphics — HLE Study & Reference

SGI Express (internal codename GR2) is the 3D graphics subsystem on the Indigo IP20. Board variants: GR2-XS, GR2-XS24, GR2-XSZ, GR2-Elan. Nobody has emulated this before.

NOT HUMAN REVIEWED. MACHINE GENERATED.

Architecture Overview

```
CPU ──GIO bus──► HQ2 ──FIFO──► GE7×1-8 ──internal bus──► RE3 ──► VRAM ──► VC1 ──► XMAP5×5 ──► Bt457×3 ──► RGB
                  │              │                        │              │
              24-bit ucode   128-bit ucode           Rasterizer     Video timing
              command FIFO   VLIW engine             pipeline       + cursor/DID
              dispatch       transforms/lighting
```

Key insight: The CPU never talks to RE3 directly. All rendering goes through the FIFO token interface: `base→fifo[TOKEN] = DATA`. The HQ2 microcode dispatches tokens to GE7, which runs geometry microcode that generates RE3 raster commands. For HLE, we intercept at the FIFO token level and skip the microcode entirely.

Custom VLSI Chips (25 total, 700,000+ gates)

Chip	Gates (SWAG)	Clock	Function
HQ2	80,000	33 MHz	Command FIFO, geometry sequencer, microcode dispatch
GE7	80,000	50 MHz	128-bit VLIW geometry/lighting engine, 32 MFLOPS each
RE3	100,000	50 MHz	Raster engine, 25–40 stage variable-depth pipeline
VC1	50,000	50 MHz	Video timing, cursor, DID, genlock
XMAP5	18,000	25 MHz	Multimode color mapper (×5 instances)
ZRB1	13,000	50 MHz	Z-buffer read/write block
RB1	2,000	50 MHz	Read block

Four GE7s execute as a **SIMD machine** with centralized sequencer. Peak aggregate: 128 MFLOPS (4 × 32).

Framebuffer Structure (1280×1024)

56 bits per pixel total, across 5-way interleaved VRAM:

- 24 bitplanes for color (24-bit full color or dual 12-bit double-buffered)
- 24-bit signed Z buffer (optional, separate memory with 2× bandwidth)
- 4 stencil bitplanes (in Z buffer low-order bits)
- 4 overlay/underlay planes
- 4 window clipping ID (CID/WID) planes

Memory Map (GIO Slot 0)

Base: `0x1F000000` (4MB window, `GR2_BASE_PHYS`)

The `struct gr2_hw` layout was proprietary (`sys/gr2hw.h` not in open-source release), but is fully reconstructible from usage across 55+ source files plus the NetBSD grtworeg.h [https://raw.githubusercontent.com/NetBSD/src/trunk/sys/arch/sgimips/gio/grtworeg.h] reverse-engineered register map.

Complete Memory Map (from NetBSD grtworeg.h)

Address Range	Size	Region
`0x1f000000 – 0x1f01ffff`	128KB	Shared data RAM (CPU↔GE communication)
`0x1f020000 – 0x1f03ffff`	128KB	(unused)
`0x1f040000 – 0x1f05ffff`	128KB	FIFO write port (indexed by token)
`0x1f060000 – 0x1f068000`	32KB	HQ2 microcode RAM
`0x1f068000 – 0x1f069fff`	8KB	GE7 instances (×8)
`0x1f06a000 – 0x1f06b004`	4KB	HQ2 registers
`0x1f06c000`		Board revision registers
`0x1f06c020`		Clock PLL
`0x1f06c040`		VC1 video controller
`0x1f06c060`		BT479 Triple-DAC (read)
`0x1f06c080`		BT479 Triple-DAC (write)
`0x1f06c0a0`		Bt457 DAC (red)
`0x1f06c0c0`		Bt457 DAC (green)
`0x1f06c0e0`		Bt457 DAC (blue)
`0x1f06c100 – 0x1f06c19f`		XMAP5 ×5 individual
`0x1f06c1a0`		XMAP5 “all” broadcast
`0x1f06c1c0`		Kaleidoscope AB1
`0x1f06c1e0`		Kaleidoscope CC1
`0x1f06c200`		RE3 registers (27-bit)
`0x1f06c280`		RE3 registers (24-bit)
`0x1f06c600`		RE3 registers (32-bit)

HQ2 Registers (offset from base ''0x1f06a000'')

Offset	Name	R/W	Description
`0x00`	`ATTRJUMP[0..15]`	RW	Attribute jump table (16 × 32-bit)
`0x40`	`VERSION`	RO	`[31:23]`=version `[22:20]`=ge_ptr `[19:16]`=ge_ctr `[13]`=fifo_low_water `[12:6]`=fifo_count `[5]`=fifo_overflow `[4:3]`=dma_sync_src `[2]`=fin1 `[1]`=fin2 `[0]`=fin3

Offset	Name	R/W	Description
0x44	NUMGE	RW	Number of GE7s (OR with HQ2_FASTSHRAMCNT_4 for 8 GEs)
0x48	FIN1	RW	Finish flag 1
0x4c	FIN2	RW	Finish flag 2
0x50	DMASYNC	WO	DMA sync source (0=vert_int, 1=vid_sync, 2=clear, 3=set)
0x54	FIFO_FULL_TIMEOUT	RW	FIFO full timeout
0x58	FIFO_EMPTY_TIMEOUT	RW	FIFO empty timeout
0x5c	FIFO_FULL	RW	FIFO high water mark (typ: 40-48)
0x60	FIFO_EMPTY	RW	FIFO low water mark (typ: 1-16)
0x64	GE7_LOAD_UCODE	RW	GE7 microcode word [25:0] (triggers load)
0x68	GEDMA	RW	GE DMA register
0x6c	HQ_GEPC	RO	HQ GE PC (hw bug: not readable)
0x70	GEPC	RW	GE program counter for ucode loading
0x74	INTR	RW	Interrupt bit (cleared by host)
0x78	UNSTALL	WO	Unstall HQ and GE (write 0 to start)
0x7c	MYSTERY	RO	Must read 0xDEADBEEF (board probe)
0x80	REFRESH	RW	Refresh register (typ: 0x10 or 0xf0)
0x100	FIN3	RW	Finish flag 3

FIFO Token Offsets (from NetBSD grtworeg.h)

FIFO base at 0x1f040000. Token numbers are word offsets into this region:

Offset from FIFO base	Token	PUC Name	Operation
0x644	0x191	PUC_INIT	Initialize GE7 for textport mode
0x648	0x192	PUC_COLOR	Set current drawing color (CI8)
0x64c	0x193	PUC_FINISH	Synchronization finish token
0x650	0x194	PUC_PNT2I	Draw single pixel
0x654	0x195	PUC_RECTI2D	Filled rectangle
0x658	0x196	PUC_CMOV2I	Move character position
0x65c	0x197	PUC_LINE2I	Draw line
0x660	0x198	PUC_DRAWCHAR	Draw character glyph
0x664	0x199	PUC_RECTCOPY	Rectangle copy/scroll
0x77c	0x1df	PUC_DATA	Data word for previous command

Reconstructed struct gr2_hw

```
struct gr2_hw {
    // HQ2 command processor registers
    struct {
        volatile uint32_t version;          // RO: version/status (see register table above)
```

```
        volatile uint32_t numge;            // RW: number of GE7s
        volatile uint32_t fin1, fin2, fin3; // RW: finish flags
        volatile uint32_t fifo_full_timeout;
        volatile uint32_t fifo_empty_timeout;
        volatile uint32_t fifo_full;        // FIFO high water mark (typ: 40-48)
        volatile uint32_t fifo_empty;       // FIFO low water mark (typ: 1-16)
        volatile uint32_t ge7loaducode;     // RW: GE7 microcode word [25:0] (triggers load)
        volatile uint32_t attrjmp[16];      // RW: attribute jump table
        volatile uint32_t gedma;            // RW: GE DMA register
        volatile uint32_t gepc;             // RW: GE program counter for ucode loading
        volatile uint32_t intr;             // RW: interrupt bit (cleared by host)
        volatile uint32_t unstall;          // WO: unstall HQ and GE (write 0 to start)
        volatile uint32_t refresh;          // RW: refresh register (typ: 0x10 or 0xf0)
        volatile uint32_t hq_gepc;          // RO: HQ GE PC (hw bug: not readable)
        volatile uint32_t mystery;          // RO: must read 0xDEADBEEF (board probe)
        volatile uint32_t dmasync;          // WO: DMA sync source
    } hq;

    volatile uint32_t hqucode[];    // HQ2 microcode RAM (24-bit words)
    volatile uint32_t fifo[];       // FIFO write port (indexed by token number)
    volatile uint32_t shram[];      // Shared RAM (32-bit words, CPU↔GE)

    struct {                        // GE7 geometry engine (×8 instances)
        volatile uint32_t ram0[];   // GE7 internal RAM (128 entries + special regs)
                                    // ram0[0xF8-0xFB]: ucode staging
                                    // ram0[0xFD]: [7:5] = GE7 revision
    } ge[8];

    struct {                        // VC1 video controller
        volatile uint16_t addrhi;   // Address high byte
        volatile uint16_t addrlo;   // Address low byte
        volatile uint16_t sram;     // SRAM read/write port
        volatile uint16_t cmd0;     // Command/data register
        volatile uint16_t cmd1;     // Command register 1
        volatile uint16_t testreg;  // Test register ([2:0] = VC1 revision)
        volatile uint16_t sysctl;   // System control
    } vc1;

    struct {                        // XMAP5 color map (×5, plus xmapall broadcast)
        volatile uint32_t misc;     // Misc (5 bytes sequential: R,G,B blackout, pgreg, cu_reg)
        volatile uint32_t mode;     // Mode registers (32 per field × 2 fields)
        volatile uint32_t clut;     // Color LUT (8192 × 3 bytes RGB, sequential)
        volatile uint32_t crc;      // CRC register
        volatile uint32_t addrlo;
        volatile uint32_t addrhi;   // [12:8] = high address bits
        volatile uint32_t bytecnt;  // Byte count (RO)
        volatile uint32_t fifostatus; // FIFO status ([0]=empty, [1]=half_full, [2]=overflow)
    } xmap[5];
    struct { /* same layout */ } xmapall; // Broadcast to all 5 XMAPs

    struct {                        // Bt457 RAMDAC (×3: reddac, grndac, bluedac)
        volatile uint8_t addr;      // Address register
        volatile uint8_t cmd2;      // Command/data register
        volatile uint8_t paltram;   // Palette RAM access
        volatile uint8_t ovrlay;    // Overlay color
    } reddac, grndac, bluedac;

    struct {                        // Board version/config registers
        volatile uint8_t rd0;       // Clock select / board rev (inverted)
        volatile uint8_t rd1;       // Config: [7]=soft_reset [6]=vc1_not_reset [5]=expanded_bkend
        volatile uint8_t rd2;       // Monitor type
        volatile uint8_t rd3;       // VRAM presence
    } bdvers;

    struct {
        volatile uint8_t write;     // Clock PLL programming (serial bit-bang)
    } clock;
};
```

RE3 Raster Engine Registers

RE3 registers are at three address regions. The full register set is not documented in the open-source

release (`re3equ.h` was proprietary), but the MAME RE2 emulation [https://github.com/mamedev/mame/blob/master/src/mame/sgi/sgi_re2.h] provides the predecessor register set. RE3 is an evolution of RE2, and most registers are expected to carry over:

RE2 Register Map (from MAME — likely basis for RE3)

Buffered Registers (write only unless noted)

Reg	Name	Bits	Description
`0x04`	ENABRGB	1	Enable 8-bit RGB mode (RE2 only)
`0x05`	BIGENDIAN	1	Enable big-endian mode
`0x06`	FUNC	4	Raster operation function
`0x07`	HADDR	2	Starting pixel location
`0x08`	NOPUP	1	Size of uaux
`0x09`	XYFRAC	4	Initial XY fraction
`0x0a`	RGB	27	Initial color values
`0x0b`	YX	22	Initial Y and X position
`0x0c`	PUPDATA	2	PUP data
`0x0d`	PATL	16	Pattern mask low
`0x0e`	PATH	16	Pattern mask high
`0x0f`	DZI	24	Delta Z integer
`0x10`	DZF	14	Delta Z fraction
`0x11`	DR	24	Delta red
`0x12`	DG	20	Delta green
`0x13`	DB	20	Delta blue
`0x14`	Z	24	Initial Z integer
`0x15`	R	23	Initial red
`0x16`	G	19	Initial green
`0x17`	B	19	Initial blue
`0x18`	STIP	16	Stipple pattern (RW)
`0x19`	STIPCOUNT	8	Stipple repeat (RW)
`0x1a`	DX	16	Delta X
`0x1b`	DY	16	Delta Y
`0x1c`	NUMPIX	11	Pixel count
`0x1d`	X	12	Initial X
`0x1e`	Y	11	Initial Y
`0x1f`	IR	3	Instruction register (triggers operation)

Unbuffered Registers (write only unless noted)

Reg	Name	Bits	Description
0x20	RWDATA	32	Read/write data (RW)
0x21	PIXMASK	24	Pixel write mask
0x22	AUXMASK	9	Auxiliary plane mask
0x23	WIDDATA	4	Window ID data
0x24	UAUXDATA	4	Micro-aux data
0x25	RWMODE	3	R/W mode: 0=FB, 1=PUP, 2=UAUX, 3=ZB, 4=WID, 6=FB_port, 7=ZB_port
0x26	READBUF	1	Buffer select for read
0x27	PIXTYPE	2	Pixel type
0x28	ASELECT	6	Anti-alias select
0x29	ALIGNPAT	1	Pattern alignment
0x2a	ENABPAT	1	Enable pattern mask
0x2b	ENABSTIP	1	Enable stipple
0x2c	ENABDITH	1	Enable dithering
0x2d	ENABWID	1	Enable WID check
0x2e	CURWID	4	Current WID
0x2f	DEPTHFN	4	Depth (Z) compare function
0x30	REPSTIP	8	Stipple repeat count
0x31	ENABLWID	1	Enable line WID
0x32	FBOPTION	2	Framebuffer option
0x33	TOPSCAN	18	First row/column
0x34	TESTMODE	—	Test mode
0x35	TESTDATA	—	Test data
0x36	ZBOPTION	1	Z buffer option
0x37	XZOOM	8	X zoom factor
0x38	UPACMODE	2	Packing mode
0x39	YMIN	11	Bottom screen mask (clip)
0x3a	YMAX	11	Top screen mask (clip)
0x3b	XMIN	12	Left screen mask (clip)
0x3c	XMAX	12	Right screen mask (clip)
0x3d	COLORCMP	1	Z compare source
0x3e	MEGOPTION	1	VRAM size option

RE2/RE3 Instruction Register (IR) Commands

IR	Name	Description
1	SHADED	Draw shaded (Gouraud) span
2	FLAT	Draw 1×5 flat span

IR	Name	Description
3	FLAT4/BLOCKWRITE	Draw 1×20 flat span or block write
4	TOPLINE	Draw top of anti-aliased line
5	BOTLINE	Draw bottom of anti-aliased line
6	READBUF	Read framebuffer
7	WRITEBUF	Write framebuffer

HLE Strategy

Why HLE, not LLE

The Express pipeline has three microprocessors (HQ2, GE7×N, RE3) running proprietary microcode. LLE would require:

- A GE7 128-bit VLIW ISA disassembler/emulator
- An HQ2 24-bit microcode emulator
- RE3 pipeline stage emulation

None of this is documented. The compiled microcode blobs are in the tree but with undocumented instruction encodings.

HLE intercepts at the CPU bus interface — the only boundary that's fully documented.

Phase 1: PROM Textport (minimal)

The PROM uses only these FIFO tokens (from `gr2_tp.c`):

Token	Value	Args	Operation
`PUC_INIT`	`0x191`	board_rev_class	Initialize GE7 for textport mode
`PUC_COLOR`	`0x192`	color_index	Set current drawing color (CI8)
`PUC_FINISH`	`0x193`	—	Synchronization
`PUC_PNT2I`	`0x194`	x, y (via PUC_DATA)	Draw single pixel
`PUC_RECTI2D`	`0x195`	x1, y1, x2, y2 (via PUC_DATA)	Filled rectangle
`PUC_CMOV2I`	`0x196`	x, y (via PUC_DATA)	Move character position
`PUC_LINE2I`	`0x197`	—	Draw line
`PUC_DRAWCHAR`	`0x198`	xsize, ysize, sper, xorig, yorig, xmove, ymove, bitmap…	Draw character glyph
`PUC_RECTCOPY`	`0x199`	length, lines, x1, y1, width, height, x2, y2	Rectangle copy/scroll
`PUC_DATA`	`0x1df`	value	Data word for previous command

Implementation:

1. Software framebuffer (1280×1024 × 8bpp CI or 24bpp RGB)
2. Token state machine parsing FIFO writes
3. Implement each token as direct framebuffer operation

4. Color LUT from XMAP CLUT writes
5. Dump framebuffer to PPM/PNG periodically or on vsync

Hardware probe requirements:

- `base→hq.mystery` must return `0xDEADBEEF`
- `base→hq.version` must return valid version (bits 31:23)
- `base→ge[i].ram0[]` must be read/write (GE count probe)
- `base→bdvers.rd0-rd3` must return board config
- Microcode download must accept writes and verify reads

Phase 2: X Server / Window System

When IRIX boots fully, the X server:

1. Downloads **new** GL microcode to HQ2/GE7 (replacing PROM textport ucode)
2. Uses a much larger token set defined in `gecmds.h` (NOT in this source release)
3. Drives RE3 through GE7 for all rendering

The GL token set includes commands for:

- Vertex submission (v2f, v3f, v4f, v2i, v3i, v4i — position + color + normal)
- Matrix operations (loadmatrix, multmatrix, pushmatrix, popmatrix)
- Lighting (lmdef, lmbind, n3f)
- Attribute setting (color, linewidth, pointsize, linestyle)
- Drawing primitives (bgnpolygon, endpolygon, bgnline, endline, bgntmesh)
- Pixel operations (rectcopy, lrectwrite, lrectread, readpixels)
- Texture (texbind, texdef, tevdef, tevbind)
- Framebuffer control (frontbuffer, backbuffer, swapbuffers, clear, zclear)

Phase 3: Full 3D GL Pipeline

Translate GL tokens → modern GPU API (or software rasterizer):

- Vertex transforms via matrices (software or GPU compute)
- Flat/Gouraud shading
- Z-buffer (24-bit)
- 8/12/24-bit color modes
- Overlay/underlay planes (4-bit aux)
- Window ID (WID/CID) planes
- Stipple patterns
- Anti-aliased lines/points
- Texture mapping (limited — RE3 has basic texture support)

VC1 Video Controller

VC1 is the **same chip** used in both GR1 and GR2. MAME has a complete VC1 emulation [https://github.com/mamedev/mame/blob/master/src/mame/sgi/vc1.h] for GR1 which is directly reusable.

Registers accessed via address/data pairs through `vc1.addrhi`, `vc1.addrlo`, `vc1.cmd0`.

Video Timing Group

Offset	Name	Bits	Description
`0x00`	VID_EP	16	Video entry pointer (into SRAM timing table)
`0x02`	VID_LC	12	Line count

Offset	Name	Bits	Description
0x04	VID_SC	10	?
0x06	VID_TSA	7	Timing strobe A
0x07	VID_TSB	7	Timing strobe B
0x08	VID_TSC	7	Timing strobe C
0x09	VID_LP	16	Line pointer
0x0B	VID_LS_EP	16	Line state entry pointer
0x0D	VID_LR	8	Line repeat count
0x10	VID_FC	32	Frame count
0x14	VID_ENAB	6	Video enable bits

Cursor Group

Offset	Name	Bits	Description
0x20	CUR_EP	16	Cursor entry pointer (into SRAM bitmap)
0x22	CUR_XL	11	Cursor X position
0x24	CUR_YL	12	Cursor Y position
0x26	CUR_MODE	8	Cursor mode
0x27	CUR_BX	8	Cursor ?
0x28	CUR_LY	11	Cursor line Y
0x2A	CUR_YC	12	Cursor Y counter (read for vblank sync)
0x2E	CUR_CC	8	Cursor color control
0x30	CUR_RC	12	Cursor repeat count

DID Group

Offset	Name	Bits	Description
0x40	DID_EP	16	DID entry pointer
0x42	DID_END_EP	16	DID end entry pointer
0x44	DID_HOR_DIV	8	Horizontal divide
0x45	DID_HOR_MOD	3	Horizontal modulus
0x46	DID_LP	16	DID line pointer
0x48	DID_LS_EP	16	DID line state entry pointer
0x4A	DID_WC	16	DID write count
0x4C	DID_RC	16	DID read count

XMAP Control Group

Offset	Name	Bits	Description

Offset	Name	Bits	Description
`0x60`	BLKOUT_EVEN	8	Blackout even field
`0x61`	BLKOUT_ODD	8	Blackout odd field
`0x62`	AUXVIDEO_MAP	8	Aux video mapping

VC1 SRAM Layout

Base	Name	Content
`0x0000`	VIDTIM_LST	Line State Table — video timing microcode
`0x0400`	VIDTIM_CURSLST	Cursor LST variant
`0x0800`	VIDTIM_FRMT	Frame Table — frame-level timing
`0x0900`	VIDTIM_CURSFRMT	Cursor frame timing
`0x09f0`	INTERLACED	Interlace flag
`0x09f2`	SCREENWIDTH	Screen width
`0x09f4`	NEXTDID_ADDR	Next DID address
`0x0a00`	CURSOR0	Cursor bitmap (32×32×2)
`0x0b00`	DID_FRMT	DID per-frame entries
`0x8000`	DID_LST_END	DID line state end

VC1 System Control

Bit	Name	Description
0	INTERRUPT	Interrupt enable
1	VTG	Video timing generator enable
2	VC1	Enable VC1
3	DID	Enable DID
4	CURSOR	Enable cursor
5	CURSOR_DISPLAY	Show cursor
6	GENSYNC	Genlock sync
7	VIDEO	Video output enable

Write `sysctl = 0x01` to disable (blackout), `sysctl = 0xFF` typical full enable.

XMAP5 Mode Register Format

Each mode register is 32 bits (from `xmaptest1.c`):

```
// Byte 3 (bits 31:24)
PIX_MODE(val)   = (val & 0x7) << 24    // Pixel format: 0-1=4bit, 2-3=8bit, 4-5=12bit, 6=16bit, 7=24bit
PIX_PG(val)     = (val & 0x1F) << 27   // Pixel page (DID selection)

// Byte 2 (bits 23:16)
PD_MODE(val)    = (val & 0x3) << 16    // 0=color_index, 1=RGB, 2=CI_immediate, 3=blackout
ALT_VIDEO(val)  = (val & 0x3) << 18   // Alternate video mode
OLAYEN(val)     = (val & 0xF) << 20   // Overlay enable (4 planes)
```

```
// Byte 1 (bits 15:8)
ULAYEN(val)     = (val & 0x1) << 8    // Underlay enable
AUX_PG(val)     = (val & 0x1F) << 9   // Aux plane page
POU_OVER(val)   = (val & 0x1) << 14   // POU overlay
POU_PX(val)     = (val & 0x1) << 15   // POU pixel

// Byte 0 (bits 7:0)
POU_PG(val)     = val & 0x1F          // POU page
POU_MODE(val)   = (val & 0x3) << 5    // POU mode: 0=off, 1=8bit, 2-3=4bit
POU_MM(val)     = (val & 0x3) << 7    // POU misc mode
```

XMAP5 register addresses (individual, spaced 0x20 apart starting at `0x6c100`):

Offset	Name	Description
`0x00`	MISC	Misc registers (5 bytes sequential)
`0x04`	MODE	Mode register array
`0x08`	CLUT	Color LUT (8192 RGB entries, sequential byte write)
`0x0c`	CRC	CRC register
`0x10`	ADDRLO	Address low
`0x14`	ADDRHI	Address high `[12:8]`
`0x18`	BYTECOUNT	Byte count (RO)
`0x1c`	FIFOSTATUS	`[0]`=empty `[1]`=half_full `[2]`=overflow

FIFO status constants: `GR2_FIFO_EMPTY=0x1`, `GR2_FIFO_HALF_FULL=0x2`, `GR2_FIFO_OVF=0x4`. FIFO depth = 512.

Bt457 DAC

Three separate DACs for R, G, B channels. Each has 256-entry gamma ramp (`paltram`) and overlay palette (`ovrlay`).

Register	Description
`addr`	Address register
`cmd2`	Command: 5:1 mux, RAM enable, overlay enable
`paltram`	Palette RAM (256 entries)
`ovrlay`	Overlay color palette

Board Revision Registers

From NetBSD grtwo.c [https://raw.githubusercontent.com/NetBSD/src/trunk/sys/arch/sgimips/gio/grtwo.c] board probe:

```
// Board revision (rd0) — value is inverted
boardrev = (~read(GR2_REVISION_RD0)) & 0x0F;

// Rev >= 4 (Elan):
backendrev  = ~read(RD1) & 0x03;
zbuffer     = read(RD1) & 0x20;         // bit 5
depth       = (read(RD1) & 0x10) ? 24 : 8;  // bit 4
monitor     = (read(RD0) & 0xF0) >> 4;

// Rev < 4 (older XS/XS24/XSZ):
backendrev  = ~(read(RD2) & 0x0C) >> 2;
zbuffer     = !(read(RD1) & 0x0C);
```

```
// VRAM slot presence (rd3): VMA=0x03, VMB=0x0C, VMC=0x30 (both bits set = empty)
depth       = 8 * (slots_present);       // 1 slot=8bpp, 2=16bpp, 3=24bpp
```

GE7 Hardware Details

- **ISA**: 128-bit VLIW (103 bits used, verified by `ram0[0xFB] & 0x7F` mask)
- **Microcode RAM**: 64K entries × 128 bits
- **Internal RAM**: 128 × 32-bit entries per GE (`ram0[0..127]`)
- **Special registers**: `ram0[0xF8-0xFB]` = ucode staging, `ram0[0xFD]` = revision
- **Shared RAM**: External to GEs, CPU-accessible, used for communication
- **FPU**: IEEE 754 single-precision (add, sub, multiply verified by `ge7diag.c`)
- **Seed ROM** + **Square Root ROM**: Hardware lookup tables
- **Sequencer**: Supports conditional execution, branching
- **FIFO input**: Receives tokens + data from HQ2 dispatch
- **RE3 output**: Drives raster engine via internal bus
- **GE7 revision register**: `0x680fc`, bits `[7:4]` = revision

Microcode Format (mcout_t)

```
// Magic numbers
#define HQ2_MAGIC  0x965e
#define GE7_MAGIC  0x966e

typedef struct {
    uint16_t f_magic;       // HQ2_MAGIC or GE7_MAGIC
    uint16_t pad1;
    uint32_t pad2;
    uint32_t f_codeoff;     // Byte offset to first code record
    uint32_t length;        // Bytes per microword (HQ2=4, GE7=20)
    uint32_t pad3;
    char version[20];
    char date[20];
    char path[20+pad];
    uint32_t data[512];     // HQ2: attrjmp[16] then padding
                            // GE7: shared RAM constants (512 words)
} mcout_t;

// HQ2 microcode record
typedef struct {
    uint16_t address;       // Microcode address (0xFFFF = end sentinel)
    uint16_t ucode[2];      // 32-bit microword as 2×16-bit (big-endian)
} HQ_RECORD;

// GE7 microcode record
typedef struct {
    uint16_t address;       // Microcode address (0xFFFF = end sentinel)
    uint16_t pad;
    uint16_t ucode[10];     // 160-bit microword as 10×16-bit
                            // ucode[0:1] → ge7loaducode (bits 25:0, triggers write)
                            // ucode[2:3] → ram0[0xFB] (bits 102:96, only 7 bits valid)
                            // ucode[4:5] → ram0[0xFA] (bits 95:64)
                            // ucode[6:7] → ram0[0xF9] (bits 63:32)
                            // ucode[8:9] → ram0[0xF8] (bits 31:0)
} GE_RECORD;
```

Board Probe Sequence

From `gr2_init.c`, the PROM's `Gr2Probe()` and `Gr2InitInfo()` execute:

1. Read `base→hq.mystery` — must return `0xDEADBEEF` (board present)
2. Read `base→bdvers.rd0` — board revision (inverted, masked to 4 bits)
3. Read `base→bdvers.rd1` — VB version, zbuffer detect
4. Read `base→bdvers.rd2` — monitor type (partial)
5. Read `base→bdvers.rd3` — VRAM bank presence (8/16/24 bit detect)

6. Probe GE7 count: write/read patterns to `base→ge[1..7].ram0[0,31,63,95,127]`
7. Start clock (PLL programming via `base→clock.write` + `base→bdvers.rd0/rd1`)
8. Init DACs (Bt457 — gamma ramp, overlay palette)
9. Init VC1 (reset, load video timing SRAM, load DID table, load cursor, enable)
10. Init XMAP (mode regs, CLUT, misc regs — via xmapall broadcast)
11. Init HQ2 regs (numge, refresh, fifo thresholds)
12. Probe chip revisions (HQ2, GE7, VC1)
13. Download HQ2 microcode (attrjmp table + ucode records)
14. Download GE7 microcode (shared RAM constants + ucode records)
15. Clear GE7 ucode RAM (64K entries)
16. Load GE7 microcode records (set gepc, write ram0[0xF8-0xFB], trigger via ge7loaducode)
17. Verify readback
18. Set `gepc=0`, write `unstall=0` (start execution)
19. Send `PUC_INIT` token via FIFO
20. Write `shram[TP_PROBE_ID] = UCODE_TP` (textport ready flag)

Diagnostic Token List (diagcmds.h)

These tokens are used with the **diagnostic** microcode (not GL microcode):

```
Token 1  DIAG_FINISH2       Set finish flag 2
Token 2  DIAG_GERAM12       GE7 internal RAM test
Token 3  DIAG_GESHRAM       GE7→shared RAM test
Token 4  DIAG_SEEDROM       Seed ROM readout
Token 5  DIAG_SQROM         Square root ROM readout
Token 6  DIAG_REINIT        RE3 register initialization
Token 7  DIAG_GEBUS         GE7 inter-bus test
Token 8  DIAG_GEFIFO        GE7→RE3 FIFO test (draws colored points)
Token 9  DIAG_REDMA         Host→RE3 DMA setup
Token 10 DIAG_SHRAMDMA      Host→SharedRAM DMA
Token 11 DIAG_SHRAMREDMA    SharedRAM→RE3 DMA
Token 12 DIAG_GESEQ         GE7 sequencer test
Token 13 DIAG_ATTR          Attribute test
Token 14 DIAG_ATTR_ALL      Attribute broadcast test
Token 15 DIAG_ACTATTR       Active attribute test
Token 16 DIAG_RELVCTR       Relative vector test
Token 19-24 DIAG_S4F..S2F   Vertex format tests (4/3/2 component, float/int)
Token 25-31 DIAG_C4I..CFLT  Color format tests
Token 33 DIAG_ONE_QUAD      Single quad rendering
Token 34 DIAG_FOUR_QUAD     Four quad rendering
Token 35 DIAG_CLEAR_SCREEN  Screen clear
Token 49 DIAG_SETCTX        Set graphics context
Token 50 DIAG_CMOV          Cursor move
Token 51 DIAG_DRAWCHAR      Character drawing
Token 52 DIAG_ZDRAW         Z-buffer draw
Token 53 DIAG_READSOURCE    Select read source (0=bitplane, 1=aux, 2=z, 3=cid)
Token 55 DIAG_VTX           Vertex submission
Token 56 DIAG_REDMA_ZB      Host→RE3 DMA with Z-buffer
Token 57 DIAG_FPUCOND       FPU condition code test
Token 59 DIAG_FLOATOPS      Float ops (IADD/ISUB/IMPY/FADD/FSUB/FMPY)
Token 60 DIAG_NORM_QUAD     Normal-lit quad
Token 67 DIAG_AUXPLANE      Aux plane test
Token 68 DIAG_CIDPLANE      CID plane test
Token 511 DIAG_CTXSW        Context switch
```

Reversing gecmds.h from IRIX Binaries

The full GL token set is in `gecmds.h` which was not released. Options:

1. **Extract from IRIX install media**: `libgl_s.a` or `/usr/gfx/libgl/EXPRESS/libGL.so` contains token numbers as immediate values in FIFO write code paths. Disassemble with MIPS toolchain.
2. **Extract from PROM binary**: The IP20 PROM contains the textport microcode with embedded token table. The `arcshqucout.h` header has the attribute jump table mapping tokens to HQ2 microcode addresses.

3. **Trace from emulation**: Once CPU is running, boot IRIX and trace all writes to `0x1F000000-0x1F3FFFFF`. Classify by access pattern.
4. **Cross-reference with GR1/Newport**: MAME's existing SGI graphics emulation [https://github.com/mamedev/mame/tree/master/src/mame/sgi] (GR1 for Personal Iris, Newport/XL for Indy) may share token naming conventions.

IRIX Source Files Reference

In this release (available)

Initialization & Boot

- `stand/arcs/lib/libsk/graphics/EXPRESS/gr2_init.c` — Full init sequence (1326 lines)
- `stand/arcs/lib/libsk/graphics/EXPRESS/gr2_tp.c` — Textport driver (token usage)
- `stand/arcs/lib/libsk/graphics/EXPRESS/vidtim.h` — Video timing tables (60Hz, 72Hz, flat panel)

Diagnostics (rich hardware interface docs)

- `stand/arcs/ide/fforward/graphics/EXPRESS/hq2regs.c` — HQ2 register map + version bitfield decode
- `stand/arcs/ide/fforward/graphics/EXPRESS/hqdiag.c` — HQ2 command tests (39KB)
- `stand/arcs/ide/fforward/graphics/EXPRESS/ge7diag.c` — GE7 tests, FPU verify, FIFO exercise
- `stand/arcs/ide/fforward/graphics/EXPRESS/ge7dma.c` — GE7 DMA tests
- `stand/arcs/ide/fforward/graphics/EXPRESS/vc1tools.c` — VC1 register names + bit widths
- `stand/arcs/ide/fforward/graphics/EXPRESS/xmaptest1.c` — XMAP5 mode/CLUT/FIFO tests
- `stand/arcs/ide/fforward/graphics/EXPRESS/bt457.c` — DAC register access
- `stand/arcs/ide/fforward/graphics/EXPRESS/bitp.c` — Bitplane read/write tests
- `stand/arcs/ide/fforward/graphics/EXPRESS/db.c` — Double-buffer / framebuffer tests
- `stand/arcs/ide/fforward/graphics/EXPRESS/RAMtools.c` — VRAM testing (52KB)
- `stand/arcs/ide/fforward/graphics/EXPRESS/dwnload.c` — Microcode download + verify
- `stand/arcs/ide/fforward/graphics/EXPRESS/dmasetup.c` — VDMA setup
- `stand/arcs/ide/fforward/graphics/EXPRESS/bdvers.c` — Board version, clock, reset
- `stand/arcs/ide/fforward/graphics/EXPRESS/gr2_pon.c` — Power-on self test

Headers

- `stand/arcs/ide/fforward/graphics/EXPRESS/diagcmds.h` — Diagnostic FIFO token definitions (98 tokens)
- `stand/arcs/ide/fforward/graphics/EXPRESS/diag_glob.h` — GE7 shared RAM layout
- `stand/arcs/ide/fforward/graphics/EXPRESS/xmap.h` — Pixel mode definitions
- `stand/arcs/ide/fforward/graphics/EXPRESS/gr2_loc.h` — Chip locations on PCB
- `stand/arcs/ide/fforward/graphics/EXPRESS/gr2.h` — Function prototypes
- `stand/arcs/ide/fforward/graphics/EXPRESS/hqucout.h` — Compiled HQ2 diagnostic microcode (6K words)
- `stand/arcs/ide/fforward/graphics/EXPRESS/geucout.h` — Compiled GE7 diagnostic microcode (30K words)

Kernel

- `irix/kern/sys/invent.h` — Board type enumeration (`INV_GR2`, `GR2_TYPE_*`)
- `irix/kern/sys/sgigsc.h` — Graphics syscall interface, context switch structures
- `irix/kern/io/vdma.c` — Virtual DMA for graphics context switching

NOT in this release (from census)

Critical missing headers (the keys to full 3D)

- `gfx/include/EXPRESS/gl/gecmds.h` (rev 1.90) — **Full GL command token definitions**
- `gfx/include/EXPRESS/gl/ge7_glob.h` (rev 1.98) — GE7 state definitions
- `gfx/include/EXPRESS/include/re3equ.h` (rev 1.13) — **RE3 register equates**
- `gfx/include/EXPRESS/include/hq2ge7.h` (rev 1.92) — HQ2↔GE7 command interface
- `gfx/include/EXPRESS/gl/imattrib.h` (rev 1.69) — Immediate mode attribute tokens
- `gfx/include/EXPRESS/gl/imdraw.h` (rev 1.31) — Drawing command tokens
- `gfx/include/EXPRESS/gl/immatrix.h` (rev 1.29) — Matrix operation tokens
- `gfx/include/EXPRESS/gl/lighting.h` (rev 1.16) — Lighting model tokens
- `gfx/include/EXPRESS/gl/texture.h` (rev 1.8) — Texture tokens
- `gfx/include/EXPRESS/gl/mc.out.h` (rev 1.5) — Microcode output format struct
- `gfx/include/EXPRESS/x/xgecmds.h` (rev 1.40) — X server command tokens

Microcode source (custom .uc assembler format)

- `gfx/EXPRESS/asm/ge7.mdl` — GE7 hardware model definition
- `gfx/EXPRESS/asm/hq2.mdl` — HQ2 hardware model definition
- `gfx/EXPRESS/promucode/ge7/prom.uc` — PROM GE7 microcode source
- `gfx/EXPRESS/promucode/hq2/prom.uc` — PROM HQ2 microcode source
- `gfx/EXPRESS/diagucode/ge7/ge7_diag.uc` — Diagnostic GE7 microcode source
- `gfx/EXPRESS/diagucode/hq2/ge7cmds.uc` — HQ2→GE7 command dispatch

SGI's own software simulator

- `gfx/EXPRESS/sim/ge7/exec.c` (rev 1.54) — GE7 instruction execution
- `gfx/EXPRESS/sim/ge7/reout.c` — RE3 output model
- `gfx/EXPRESS/sim/ge7/iobuf.c` — I/O buffer simulation
- `gfx/EXPRESS/sim/hq2/exec.c` (rev 1.23) — HQ2 instruction execution
- `gfx/EXPRESS/sim/ge7/ge7.h` — GE7 simulator data structures
- `gfx/EXPRESS/sim/hq2/hq2.h` — HQ2 simulator data structures

GR1→GR2 Evolution (from MAME GR1 Emulation)

MAME has complete GR1 (Eclipse) emulation [https://github.com/mamedev/mame/tree/master/src/mame/sgi]. GR2 (Express) is its direct successor with the same architecture. Nearly everything carries over.

Component Mapping

Component	GR1 (Eclipse)	GR2 (Express)	Delta
Command processor	HQ1 (custom sequencer)	HQ2 (24-bit ucode)	HQ2 adds `attrjmp[16]` dispatch table
Geometry engine	GE5 ×1 (64-bit ucode + WTL3132 FPU)	GE7 ×1-8 (128-bit VLIW, integrated FPU)	SIMD, wider datapath
Raster engine	RE2 (64 registers)	RE3 (deeper pipeline, 25-40 stages)	Extended register set at 3 address regions
Color mapper	XMAP2 ×5	XMAP5 ×5	More mode register bits (32-bit vs 16-bit)
Video controller	VC1	VC1	**Same chip**
DAC	Bt457 ×3	Bt457 ×3	**Same chips**
Cursor	Bt431 ×2 (separate chips)	VC1 internal	Cursor moved into VC1
FIFO depth	512 entries	512 entries	Same

FIFO Dispatch — Same Mechanism

GR1: CPU writes to FIFO region. GE5 `fetch` reads 64-bit word where upper bits = token, lower = data. Token indexes jump table: `pc = (bus » 31) & 0x1fe`.

GR2: CPU writes to `base→fifo[TOKEN] = DATA`. Address encodes token. HQ2 dispatches via `attrjmp[16]` → microcode → GE7.

Same concept: token dispatches to microcode routine.

GR1 GL Token Set — The Rosetta Stone

The GE5 source (`sgi_ge5.cpp`) contains `token_gl[]` with **~150 named GL tokens**. These are the GR1 equivalents of the missing `gecmds.h`. Token **names** and **semantics** should be identical for GR2 — only **numbers** change per hardware generation, because the IRIS GL API is fixed.

GR1 GL Tokens (from MAME sgi_ge5.cpp token_gl[]):

#	Token	GL Function
0	GE_DATA	Data word for previous command
1	GE_INIT	Initialize
10	GE_COLOR	Set color (indexed)
11	GE_FINISH0	Synchronization
12	GE_PNT2I	Point 2D integer
13	GE_SBOXI	Filled rectangle (sbox)
14	GE_CMOV2I	Character move 2D integer
15	GE_DRAWCHAR	Draw character glyph
16	GE_HAND	Handshake
17	GE_FBOPT	Framebuffer option
18	GE_ZBOPT	Z-buffer option
19	GE_TOPSCAN	Top scan line
20	GE_READPIXELS	Read pixel data
21	GE_WRITEPIXELS	Write pixel data
22	GE_LOADRE	Load RE register directly
23	GE_GETCPOS	Get character position
24	GE_PICKMODE	Enter pick mode
25	GE_PIXTYPE	Set pixel type
26	GE_PIXWRITEMASK	Pixel write mask
27	GE_POPNAME	Pop name (picking)
28	GE_PUSHNAME	Push name (picking)
29	GE_READBLOCK	Read block
30	GE_RECTREAD	Rectangle read
31	GE_READBUF	Read buffer select
32	GE_READPIXDMA	Read pixels via DMA
33	GE_AUXWRITEMASK	Aux plane write mask
34	GE_READRGB	Read RGB
35	GE_RECTCOPY	Rectangle copy
36	GE_RGBCOLOR	RGB color (direct)
37	GE_RGBSHADERANGE	RGB shade range
38	GE_RWMODE	Read/write mode
39	GE_SCREENCLEAR	Screen clear
40	GE_SHADEMODEL	Flat/Gouraud
41	GE_SHADERANGE	Shade range
42	GE_WRITEBLOCK	Write block
43	GE_RECTWRITE	Rectangle write
44	GE_WRITEPIXDMA	Write pixels via DMA
45	GE_BEGINBBOX	Begin bounding box
46	GE_ZBUFFER	Enable/disable Z
47	GE_ZCLEAR	Clear Z buffer
48	GE_ZOOMFACTOR	Zoom factor
49	GE_READSOURCE	Read source select
50	GE_DRAWMODE	Draw mode
51	GE_CZCLEAR	Color+Z clear
53	GE_ZFUNCTION	Z compare function
55	GE_FLATMODE	Flat shade mode
56	GE_LMCOLOR	Lighting material color
57	GE_LOADAMBIENT	Load ambient
58	GE_DEPTHFN	Depth function
59	GE_LOADDIFFUSE	Load diffuse

```
 60  GE_LOADMATRIX          Load 4x4 matrix
 61  GE_MULTMATRIX          Multiply matrix
 62  GE_PUSHMATRIX          Push matrix stack
 63  GE_POPMATRIX           Pop matrix stack
 64  GE_LOADSPECULAR        Load specular
 65  GE_LOADEMISSION        Load emission
 71  GE_LOADVIEWP           Load viewport
 72  GE_POLYGON             Begin polygon
 73  GE_ENDPOLYGON          End polygon
 74  GE_TRANSLATEI          Translate integer
 75  GE_TRANSLATE           Translate float
 76  GE_LINESTYLE           Line style
 77  GE_LINEWIDTH           Line width
 78  GE_VERTEX2I            Vertex 2D integer
 79  GE_VERTEX2             Vertex 2D float
 80  GE_VERTEX3I            Vertex 3D integer
 81  GE_VERTEX3             Vertex 3D float
 82  GE_VERTEX4I            Vertex 4D integer
 83  GE_VERTEX4             Vertex 4D float
 84  GE_RVERTEX2I           Relative vertex 2D integer
 88  GE_CLOSEDLINE          Begin closed line
 89  GE_ENDCLOSEDLINE       End closed line
 91  GE_ANTIALIAS           Anti-aliasing mode
 92  GE_COLORF              Color float (c3f/c4f)
 93  GE_PNT2                Point 2D float
100  GE_MOVE2I              Move 2D integer
110  GE_DRAW2I              Draw 2D integer
111  GE_DRAW2               Draw 2D float
112  GE_DRAW3I              Draw 3D integer
113  GE_DRAW3               Draw 3D float
122  GE_ENABLWID            Enable WID
123  GE_LOADGE              Load GE microcode
129  GE_FRONTFACE           Front face mode
130  GE_BACKFACE            Back face mode
131  GE_CONCAVE             Concave polygon mode
132  GE_PATTERN             Pattern
134  GE_LOADNORMAL          Load normal matrix
137  GE_MMODE               Matrix mode
138  GE_NORMAL              Normal vector (n3f)
140  GE_LIGHTATTR1          Light attributes
143  GE_BINDLIGHT           Bind light (lmbind)
149  GE_BEGINMESH           Begin triangle mesh
150  GE_ENDMESH             End triangle mesh
151  GE_SWAPMESH            Swap mesh vertex
157  GE_CURRENTWID          Set current WID
160  GE_DEPTHCUE            Depth cue (fog)
166  GE_ENABDITH            Enable dithering
170  GE_SETMATRIX           Set matrix
174  GE_FEEDBACK            Begin feedback
176  GE_PASSTHROUGH         Passthrough
180  GE_SCRMASK             Screen mask (clip rect)
181  GE_ZSOURCE             Z source
184  GE_RASTEROP            Raster operation
196  GE_STRIP               Triangle strip
240  GE_CTX0                Context switch 0
255  GE_CTX1                Context switch 1
```

This gives us the **complete vocabulary** of GL commands GR2 must support. When reversing GR2 `gecmds.h` from IRIX binaries, we search for these exact function names in the FIFO write code paths.

RE2 Registers → RE3

RE2's 64-register set maps directly to RE3's three address regions (`0x1f06c200`, `0x1f06c280`, `0x1f06c600`). Key registers that carry over:

Drawing State

- `FUNC` — 4-bit raster operation (src/dst logic)
- `RGB` — packed initial color → unpacks to R, G, B
- `YX` — packed position → unpacks to X, Y

- `DR/DG/DB` — color deltas (for Gouraud shading)
- `DX/DY` — position deltas
- `DZI/DZF` — Z deltas (integer + fraction)
- `NUMPIX` — pixel count per operation
- `IR` — instruction register (write triggers drawing)

IR Instructions

IR	Name	Description
1	SHADED	Draw Gouraud-shaded span (increments R/G/B/Z per pixel)
2	FLAT	Draw 1×5 flat-shaded span
3	FLAT4/BLOCKWRITE	Draw 1×20 flat span or block write
4	TOPLINE	Top of anti-aliased line
5	BOTLINE	Bottom of anti-aliased line
6	READBUF	Read framebuffer via DMA
7	WRITEBUF	Write framebuffer via DMA

Pixel Control

- `PIXMASK` (24-bit), `AUXMASK` (9-bit) — write masks
- `WIDDATA` (4-bit), `UAUXDATA` (4-bit) — WID/aux values to write
- `RWMODE` — 0=FB, 1=PUP, 2=UAUX, 3=ZB, 4=WID, 6=FB_port, 7=ZB_port
- `PIXTYPE` — pixel type
- `DEPTHFN` — Z compare function (4-bit)
- `ENABPAT/ENABSTIP/ENABDITH/ENABWID` — feature enables

Clipping

- `XMIN/XMAX/YMIN/YMAX` — hardware clip rectangle
- Clip coords account for 5-way interleave: `actual_x = (reg » 3) * 5 + (reg & 7)`

VRAM Layout — Identical

```
Stride = 0x500 (1280 decimal)
offset = y * 0x500 + x

Pixel format (32 bits):
  [31:28] = WID (window ID, 4 bits)
  [27:24] = overlay/underlay/aux (4 bits)
  [23:16] = blue (or CI[23:16])
  [15:8]  = green (or CI[15:8])
  [7:0]   = red (or CI[7:0])

Y=0 is bottom of screen, Y=1023 is top.
5-way interleave: pixel X maps to XMAP channel (X % 5).
```

Display Pipeline (from RE2 screen_update)

```
For each pixel at (x, y):
  1. Read VRAM[y * 0x500 + x]
  2. Extract WID = pixel[31:28]
  3. Look up mode = XMAP[x % 5].mode[WID]
  4. Check overlay: if (pixel[27:24] & mode.OE) → display overlay color
  5. Check underlay: if (mode.UE && pixel[23:0] == 0) → display underlay color
  6. Otherwise decode pixel per mode.DM:
       0 = CI8 single-buffered (8-bit color index → CLUT)
       1 = CI4 double-buffered (4-bit × 2, buffer select)
```

```
      2 = CI12 double-buffered (12-bit × 2)
      5 = RGB12 double-buffered
      7 = RGB24 single-buffered (direct color)
7. Apply per-DAC gamma ramp: R=DAC0[r], G=DAC1[g], B=DAC2[b]
```

This logic is identical for GR2/XMAP5, with XMAP5 adding more mode register bits (32-bit vs XMAP2's 16-bit).

What This Means for GR2 HLE

- **Phase 1** (textport): Needs only PUC tokens — already known from IRIX source. MAME confirms the pattern.
- **Phase 2** (X server): The GR1 token vocabulary tells us exactly which ~150 commands to expect. Token numbers differ but names/semantics match.
- **Phase 3** (full 3D): RE2's drawing model (IR commands, Gouraud spans, DMA pixel read/write) is the template for RE3's software rasterizer.
- **Display output**: RE2's `screen_update` is directly portable — same VRAM layout, WID lookup, XMAP mode decode, DAC gamma.

External References

Available Online

- NetBSD grtworeg.h [https://raw.githubusercontent.com/NetBSD/src/trunk/sys/arch/sgimips/gio/grtworeg.h] — BSD-licensed reverse-engineered GR2 register map with exact byte offsets (Christopher Sekiya, 2004)
- NetBSD grtwo.c [https://raw.githubusercontent.com/NetBSD/src/trunk/sys/arch/sgimips/gio/grtwo.c] — Working wscons framebuffer driver using GR2 FIFO tokens for text console
- MAME sgi_re2.h [https://github.com/mamedev/mame/blob/master/src/mame/sgi/sgi_re2.h] — Complete RE2 register enum (RE3 predecessor, likely compatible register set)
- MAME sgi_re2.cpp [https://github.com/mamedev/mame/blob/master/src/mame/sgi/sgi_re2.cpp] — Full RE2 raster engine emulation with drawing routines
- MAME sgi_ge5.h [https://github.com/mamedev/mame/blob/master/src/mame/sgi/sgi_ge5.h] — GE5 geometry engine (GE7 predecessor), fully emulated as CPU device
- MAME sgi_xmap2.h [https://github.com/mamedev/mame/blob/master/src/mame/sgi/sgi_xmap2.h] — XMAP2 color mapper (XMAP5 predecessor), mode register format
- MAME SGI source tree [https://github.com/mamedev/mame/tree/master/src/mame/sgi] — Complete GR1 emulation (VC1 reusable; RE2/GE5/XMAP2 inform GR2 design)
- IRIX 6.5.7m source [https://github.com/calmsacibis995/irix-657m-src] — Express diagnostic and boot code
- Higher Intellect — Elan Graphics [https://wiki.preterhuman.net/Elan_Graphics] — Elan Technical Report (architectural overview, gate counts, no register-level detail)
- sgistuff.net — Express Graphics [http://www.sgistuff.net/hardware/graphics/express.html] — Hardware overview and photos
- irix7.com techpubs [https://irix7.com/techpubs.html] — Archived SGI technical publications

Search Results for Missing Headers

Comprehensive web search (2026-03-07) confirmed: none of the 7 critical missing headers exist anywhere online. Specifically:

- `gecmds.h` — 0 results on GitHub, Google, archive.org
- `re3equ.h` — 0 results anywhere
- `hq2ge7.h` — 0 results anywhere
- `ge7shram.h` — 0 results anywhere
- `gr2hw.h` — 0 results as a file (struct reconstructible from usage)
- `express.h` — 0 relevant results (common filename, no SGI GR2 hits)

- `rss.h` — 0 relevant results (common filename)

No existing GR2/Express emulation has been attempted anywhere (MAME, other emulators, FPGA projects).

Implementation Checklist

Verilog/RTL Side

- [ ] GIO slot 0 address decode (`0x1F000000`, 4MB)
- [ ] Register file for HQ2 registers (read/write with correct reset values)
- [ ] Mystery register (hardwired `0xDEADBEEF`)
- [ ] Board version registers (hardwired for Elan config)
- [ ] GE7 RAM array (8 × 256 × 32-bit, read/write)
- [ ] Shared RAM array (8K × 32-bit, read/write)
- [ ] HQ2 microcode RAM (address-indexed, read/write)
- [ ] VC1 register interface (address/data pair protocol)
- [ ] VC1 SRAM (8KB, read/write)
- [ ] XMAP register interface (5 individual + broadcast)
- [ ] XMAP CLUT (8192 × 3 bytes per XMAP)
- [ ] DAC register interface (3 × Bt457)
- [ ] FIFO write port (active — generates HLE events)
- [ ] Clock PLL serial interface (accept writes, no-op)
- [ ] RE3 register regions (27-bit, 24-bit, 32-bit at documented addresses)
- [ ] Kaleidoscope AB1/CC1 registers (stub)

C++ Cosim / HLE Side

- [ ] FIFO token state machine
- [ ] Software framebuffer (1280×1024 × 32bpp internal)
- [ ] Token handlers: PUC_INIT, PUC_COLOR, PUC_RECTI2D, PUC_PNT2I, PUC_CMOV2I, PUC_DRAWCHAR
- [ ] CLUT shadow from XMAP writes
- [ ] Framebuffer dump (PPM on demand or periodic)
- [ ] Finish flag management (for synchronization)
- [ ] VDMA support (pixel DMA read/write from/to framebuffer)
- [ ] VC1 video timing (minimal — frame counter, vblank signal)
- [ ] Cursor rendering (32×32 from VC1 SRAM)

Board Configuration for Elan (IP20)

```
// bdvers register values for GR2-Elan 24-bit with Z-buffer:
bdvers.rd0 = ~4 & 0xF;     // Board rev 4 (Elan), inverted → 0x0B
bdvers.rd1 = 0x21;          // VB version 1, zbuffer present
                             // [5]=zbuf, [4]=24bit, [1:0]=VB_rev
bdvers.rd2 = 0x00;          // Monitor type (60Hz default)
bdvers.rd3 = 0x00;          // All 3 VRAM banks present

// After board init:
hq.version = (4 << 23) | ... // HQ2 version 4
hq.mystery = 0xDEADBEEF
ge[0].ram0[0xFD] >> 5 = GE7 revision

// GE probe: 1 GE for Elan (base config), 4 for XS24, 8 for Extreme
numge = 1;  // For minimal Elan
```